On Language Model Scaling Laws with SVG Code

Vincent Hepola

Spring 2026

Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Approach . . . 2

2 Data **2**
2.1 Dataset and Preprocessing . . . 3
2.2 Tokenization . . . 3

3 Methods **4**
3.1 Model architectures . . . 4
3.2 Training setup and hyperparameters . . . 4
3.3 Experimental design for scaling studies . . . 5
3.4 Evaluation metrics . . . 5

4 Results **5**
4.1 Fixed LR vs. μ**P** comparisons and scaling . . . 5
4.2 Scaling law extrapolation prediction . . . 5
4.3 Generated samples with evaluation . . . 6
4.4 Perplexity . . . 6

5 Discussion **7**
5.1 Key insights from scaling experiments . . . 7
5.2 Design decisions and their impact . . . 7
5.3 Limitations and future work . . . 7

6 Conclusion **7**

7 References **8**

8 Appendix **8**

1 Introduction

1.1 Motivation

Vector graphics are fundamentally different from rasterized pixel images; they are resolution-independent and defined entirely by mathematical paths and shapes described in XML. This makes Scalable Vector Graphics (SVGs) an excellent testbed for sequence modeling. By training a Transformer on SVG code, we can investigate whether language models can learn rigid syntactical grammar, mathematical coordinate structures, and continuous geometric relationships simultaneously—where the output can be immediately verified through visual rendering.

1.2 Approach

We approach this problem by training a family of decoder-only Transformer language models of varying widths from scratch on a curated dataset of SVG icons. To analyze the scaling behavior of these models, we compare Standard Parameterization (SP) against Maximal Update Parameterization ($\mu\mathbf{P}$).

By establishing an empirical scaling law, we aim to demonstrate how $\mu\mathbf{P}$ enables zero-shot hyperparameter transfer, allowing us to find an optimal learning rate on a proxy model and safely scale it up to our largest architecture. Finally, we evaluate the generative capabilities of the fully trained models through conditional and unconditional SVG rendering.

2 Data

A valid SVG file consists of an XML-based text string which encodes how to draw lines and curves. Figure 1 shows an SVG image of a circled checkmark drawn in one continuous stroke. Figure 8 shows the raw SVG code for the checkmark with some text colored. The blue `svg` tags signify the beginning and end of the SVG code. The orange `path` tags enclose the arguments for a single path, while the nested purple data string `d="M10.088399887084961 5.28..."` defines how to draw the path [4]. In this sample image, there is only one path. The data strings are the most important part for our models to learn to generate.

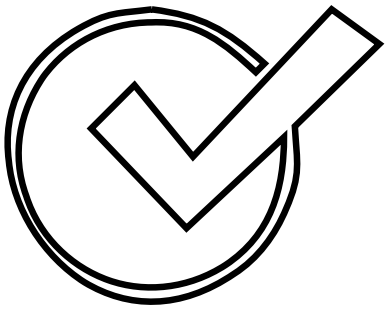


Figure 1: Checkmark SVG Image

2.1 Dataset and Preprocessing

Our training data was sourced from the `starvector/svg-stack-simple` dataset via Hugging Face [5]. To normalize the data, we applied a strict regex-based cleaning pipeline. All XML comments and metadata were stripped, and coordinate precision was rounded to a single decimal place. The string data within the `<path>` tags was spaced out to isolate individual command letters (i.e., M, L, C) from numerical coordinates. Finally, any SVGs containing fewer than 50 characters or exceeding a length threshold of 2,048 characters (defined in our code as `TOKEN_THRESHOLD`) were filtered out. This threshold was chosen to balance the complexity of the SVGs with the memory constraints of our `BLOCK_SIZE = 256` context window, ensuring the self-attention mechanism remained computationally feasible. We also included checks for broken XML encodings and SVG rendering errors, but we never experienced these during data cleaning.

Initially, we worked with the `starvector/svg-icons-simple` and `starvector/svg-emoji-simple` datasets, but their combination alone yielded fewer than 100 million tokens during the next section. Therefore, we omitted them. Figure 2 shows a size comparison of the three datasets.

Starvector Dataset	Number of SVGs
svg-icons-simple	89370
svg-emoji-simple	5106
svg-stack-simple	**1294375**

Figure 2: Number of SVG strings in considered datasets

We then divided the cleaned and filtered dataset into 98% train / 1% val / 1% test splits of SVG strings. Figure 3 shows a preprocessed version of the raw checkmark string, color-coded in the same way as in Figure 8.

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"><path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 10.1 5.3 L 10.1 5.3 C 12.4 5.6 13.5 6.2 15.3 7.8 L 14.9 8.2 C 13.5 6.9 12.2 5.8 10.1 5.9 C 8.0 5.9 6.2 6.9 5.1 8.4 C 4.0 10.0 3.6 12.0 4.3 13.9 C 4.9 15.8 6.4 17.2 8.1 17.8 C 9.8 18.4 11.8 18.2 13.5 17.1 C 15.6 15.7 16.2 13.6 16.2 11.2 L 11.7 15.4 L 7.3 10.8 L 9.3 8.8 L 12.0 12.1 L 15.2 8.7 L 18.4 5.3 L 20.6 6.9 L 16.7 10.7 C 16.8 12.4 17.0 12.9 16.3 14.5 C 15.8 15.7 15.0 16.8 13.9 17.5 C 11.4 19.2 9.0 19.1 7.0 17.9 C 5.1 16.7 3.7 14.5 3.5 12.3 C 3.2 10.0 4.1 7.7 6.7 6.2 C 8.1 5.4 8.5 5.5 10.1 5.3 L 10.1 5.3"></path></svg>

Figure 3: Cleaned Checkmark SVG string

Subset	Number of SVGs
Train	294000 (98.01%)
Val	3000 (1.00%)
Test	3000 (0.99%)
total	208784

Figure 4: Number of SVGs per Subset, after cleaning and filtering

2.2 Tokenization

We trained a custom Byte-Pair Encoding (BPE) tokenizer using the Hugging Face `tokenizers` library [1, 2]. The vocabulary size (`VOCAB_SIZE`) was set to 1,000 tokens, somewhat arbitrarily, at the

lower end of the recommended range. Both `WhitespaceSplit()` and `Punctuation()` were added to the pre-tokenizer during training. A sample of the resulting tokenizer vocabulary after training is shown in Figure 19.

The table in Figure 5 describes the sizes of each subset of our cleaned dataset and the number of tokens created after the tokenization step. Figures 9 and 10 demonstrate that all three subsets share a relatively similar distribution of string and token lengths.

Subset	Number of SVGs	Number of Tokens
Train	204630 (98.01%)	130428077 (98.00%)
Val	2088 (1.00%)	1355632 (1.02%)
Test	2066 (0.99%)	1301001 (0.98%)
Total	208784	133084710

Figure 5: SVGs and Tokens per Subset

3 Methods

3.1 Model architectures

Our CustomTransformer model class shown in figures 20–24 was built from the ground up with PyTorch following the original *Attention Is All You Need* paper and Andrej Karpathy's *nanoGPT* repository [6, 3]. Five configurations of the model were considered for our experiments. Due to μ**P** package requirements, the depth dimensions (`n_layers` and `n_heads`) were held constant across all models while width dimensions (`d_model` and `d_ff`) varied. The model configurations are shown in Figure 6.

Name	**~Params**	**d_model**	**n_layers**	**n_heads**	**d_ff**
Tiny	1.5M	128	6	6	512
Small	3M	192	6	6	768
Medium	11.5M	384	6	6	1536
Large	20M	512	6	6	2048
XL	44M	768	6	6	3072

Figure 6: Five model configurations

3.2 Training setup and hyperparameters

The models were trained using the AdamW optimizer and a cosine annealing learning rate scheduler with a linear warmup period spanning the first 5% of training steps. Training batch sizes varied from 64 to 128 depending on GPU availability. During our full training of the μ**P**-tuned "XL" model, we swapped out our manual attention calculations with PyTorch's native `F.scaled_dot_product_attention` (FlashAttention) to speed up our iteration throughput. We utilized three unique platforms during this project: An M4 Macbook Air with MPS acceleration, a Windows 10 PC with an NVIDIA RTX 3080 GPU + CUDA, and NYU HPC's Google Cloud Bursting with an NVIDIA A100 GPU + CUDA.

3.3 Experimental design for scaling studies

To analyze the scaling behavior of the Transformer, we evaluated the models across the 5 varying parameter counts outlined in Figure 6. The depth of the network (number of layers `n_layers` and attention heads `n_heads`) was kept constant, while the width (embedding dimension `d_model` and feed-forward dimension `d_ff`) was scaled. We compared the performance of models using Standard Parameterization (SP) versus Maximal Update Parameterization (μ**P**). To do this, we first performed a 1000-step learning rate sweep on the proxy "Tiny" model to identify the optimal base learning rate. Seven orders of learning rates were tested: LR = 10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, and 10^{-1}. Once the optimal learning rate was identified, we then trained each of the five models on one full pass of our training data subset, tracking both training and validation losses along the way. Using the μ**P** framework, we repeated this process again to find a potentially new optimal learning rate and apply it across all larger model sizes to observe zero-shot hyperparameter transfer and plot the resulting training and validation losses.

3.4 Evaluation metrics

The primary quantitative metric for model performance during training was the Cross-Entropy Loss on the validation dataset, representing the model's ability to accurately predict the next token in the SVG sequence.

For qualitative evaluation, we assessed the generated SVGs based on two structural criteria: *XML validity* (whether the generated string could be parsed correctly as a valid XML document using the `lxml` library) and *renderability* (whether the string could be successfully rasterized into a PNG using CairoSVG). We evaluated these metrics across batches of both unconditional and conditional generation prompts.

4 Results

4.1 Fixed LR vs. μP comparisons and scaling

Figure 25 shows that the standard fixed learning rate approach (1×10^{-2}) works fine while training smaller models, but not for the largest models we tested. Almost immediately into training one epoch, the "Large" and "XL" models' training loss curves diverge from the smaller models. We also see that validation loss does not follow a power-law relationship with respect to the model parameters, as the fitted scaling exponent $\alpha \approx 0.0001$.

In contrast, Figure 26 shows that the μ**P** framework stabilized the optimal learning rate on the "Tiny" model and successfully transferred it to the "Large" and "XL" models without causing training divergence. The validation loss follows a power-law relationship with respect to the model parameters, as the fitted scaling exponent $\alpha \approx -1.8330$.

4.2 Scaling law extrapolation prediction

When considering a model with 440M parameters—10x the parameters of our "XL" model—which is tuned with μ**P**, we would confidently predict the training loss to be quite similar at around 0.46. This prediction follows directly from the fitted power law in Figure 26.

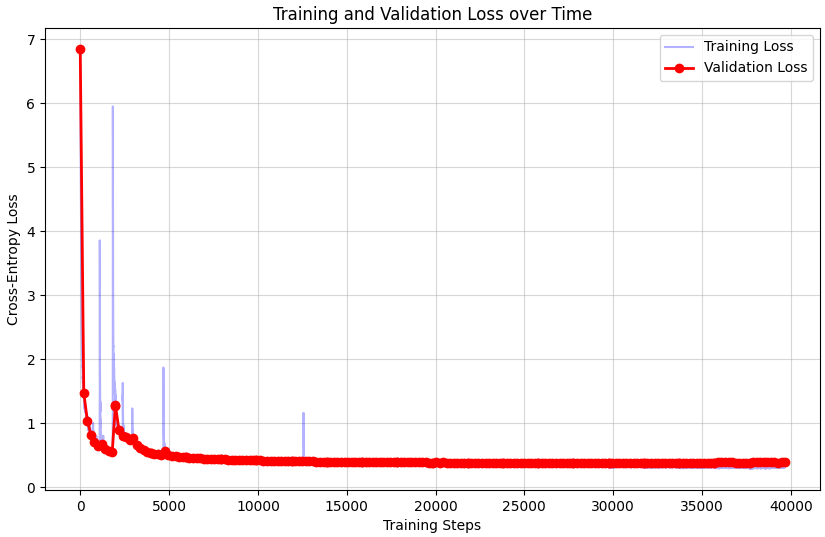


Figure 7: Training and validation loss of final "XL" model.

4.3 Generated samples with evaluation

Our fully trained "XL" model (with loss functions show in Figure 7) was evaluated by generating 10 unconditional samples and five conditional samples (Figures 12 - 16). Using temperature scaling (0.7) and Top-K filtering ($K = 5$), the model achieved a 100% success rate in both XML validity (10/10 unconditional, 5/5 conditional) and renderability. There is some hidden variability in our success however, as the model would occasionally try to generate strings longer than our `max_new_tokens = TOKEN_THRESHOLD` limit while testing.

While the model successfully learned the strict syntactic grammar of SVGs, a post-processing function was required to resolve spacing artifacts introduced by the tokenizer. After post-processing, the rendered outputs visually resembled abstract geometric squiggles rather than coherent icons. The unconditioned generation shown in Figure 13, for example, contains an unidentifiable chaotic mess of somewhat symmetric shapes.

4.4 Perplexity

The "XL" model achieved a final test Cross-Entropy loss of 0.3839, corresponding to a remarkably low Test Perplexity of 1.4681. In natural language modelling, a perplexity this low would typically indicate near-perfect mastery of the dataset's semantic meaning. However, in the SVG domain, this metric is in stark contrast with the model's inability to generate coherent images. We suspect that the model has learned the predictable syntactic transitions (e.g., spaces following coordinates) without fully grasping the global geometric structure.

Metric	Value
Test Cross-Entropy Loss	0.3839
Test Perplexity	1.4681

5 Discussion

5.1 Key insights from scaling experiments

The scaling plots in Figure 26 confirm that increasing model width reliably improves the next-token prediction capabilities on SVG data, following predictable power-law dynamics, if proper care is taken when choosing a learning rate based on model size.

The results strongly validate the Maximal Update Parameterization framework. Under Standard Parameterization, the optimal learning rate shifts as model width increases, making hyperparameter tuning for large models prohibitively expensive. However, $\mu\mathbf{P}$ allowed us to tune the learning rate cheaply on the 1.5M parameter "Tiny" model and safely transfer it to the 44M parameter "XL" model. The $\mu\mathbf{P}$ models maintained stability at learning rates that would typically cause divergence in standard Transformers.

5.2 Design decisions and their impact

Several design choices significantly impacted our training pipeline. Processing the HuggingFace datasets via PyArrow's zero-copy memory mapping prevented the CPU from bottlenecking the GPU data loaders. Furthermore, implementing FlashAttention avoided materializing the $N \times N$ attention matrix in memory, which nearly quadrupled our iteration speed and saturated the A100 GPU's memory bandwidth.

5.3 Limitations and future work

The primary limitation of this study lies in the token distribution of the training corpus. The exceptional test perplexity achieved by our "XL" model suggests that the dataset suffers from severe token imbalance, heavily dominated by formatting characters like spaces and repetitive XML tags. Future work should investigate more sophisticated coordinate quantization techniques or custom tokenizer pre-processing that normalizes numerical values. This would force the model to learn the underlying mathematical distributions rather than simply memorizing high-frequency syntactic formatting.

The $\mu\mathbf{P}$ framework required us to hold depth dimensions constant across all the models we tested, so our results are comparable when varying the width dimensions. We realized this limitation late into testing, leaving us to apply a quick fix by holding the depth dimensions constant without modifying the width dimensions. This decision resulted in model configurations that did not cover as wide of range of parameter counts as originally planned.

We also want to produce a much greater variety of model-generated examples at different temperatures and topk-choices, and subsequently provide a more thorough analysis of our models' generative behaviors.

6 Conclusion

In this project, we successfully collected, cleaned, and tokenized a dataset of over 200,000 SVG paths to train a custom Transformer architecture from scratch. We verified empirical scaling laws and demonstrated the effectiveness of Maximal Update Parameterization for zero-shot hyperparameter transfer. While generating meaningful SVG art with language models remains a challenge, our models successfully learned to natively write and format functional, renderable SVG files.

The GitHub repository for this project can be found at github.com/vHepp/ml-svg-transformers.

7 References

[1] Hugging Face. *Byte-Pair Encoding tokenization*. URL: `https://huggingface.co/learn/llm-course/en/chapter6/5`.

[2] Hugging Face. *Tokenizers*. URL: `https://huggingface.co/docs/tokenizers/index`.

[3] Andrej Karpathy. *NanoGPT*. `https://github.com/karpathy/nanoGPT`. 2022.

[4] MDN Web Docs. *SVG: Scalable Vector Graphics*. 2025. URL: `https://developer.mozilla.org/en-US/docs/Web/SVG`.

[5] Juan A. Rodriguez et al. "StarVector: Generating Scalable Vector Graphics Code from Images and Text". In: *arXiv preprint arXiv:2312.11556* (2023).

[6] Ashish Vaswani et al. "Attention Is All You Need". In: *CoRR* abs/1706.03762 (2017). arXiv: `1706.03762`. URL: `http://arxiv.org/abs/1706.03762`.

8 Appendix


```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px"
width="200px"><path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0"
d="M10.088399887084961 5.288879871368408 L10.088399887084961 5.288879871368408
C12.444925308227539 5.640568733215332 13.539755821228027 6.16114616394043
15.298320770263672 7.76855993270874 L14.915999412536621 8.200559616088867
C13.477609634399414 6.851019382476807 12.187517166137695 5.837609767913818
10.088399887084961 5.856959819793701 C8.021493911743164 5.876012802124023 6.241535663604736
6.91029167175293 5.139567852020264 8.43328857421875 C4.037599563598633 9.95628547668457
3.6136229038238525 11.968000411987305 4.258681297302246 13.941926002502441
C4.878276824951172 15.837934494018555 6.361117362976074 17.179969787597656 8.10135269165039
17.773653030395508 C9.841588973999023 18.36733627319336 11.839221000671387
18.212671279907227 13.488399505615234 17.115280151367188 C15.637887954711914
15.684974670410156 16.16459846496582 13.564998626708984 16.18391990661621 11.155439376831055
L11.701920509338379 15.421440124511719 L7.349520206451416 10.799039840698242
L9.33672046661377 8.757840156555176 L11.969759941101074 12.090719223022461
L15.19464111328125 8.704919815063477 L18.41952133178711 5.319119930267334
L20.622718811035156 6.930480003356934 L16.68071937561035 10.68239974975586
C16.761396408081055 12.41808032989502 16.974395751953125 12.875062942504883
16.29591941833496 14.52455997467041 C15.809246063232422 15.707751274108887
14.966665267944336 16.777477264404297 13.914800643920898 17.509920120239258
C11.439870834350586 19.23328399658203 8.954255104064941 19.052337646484375
7.026443958282471 17.878238677978516 C5.098632335662842 16.70414161682129 3.728624105453491
14.53689193725586 3.4849071502685547 12.287652969360352 C3.2411904335021973
10.038413047790527 4.123765468597412 7.707181930541992 6.701119899749756 6.205120086669922
C8.084009170532227 5.39918327331543 8.529239654541016 5.498373508453369 10.088399887084961
5.288879871368408 L10.088399887084961 5.288879871368408"></path></svg>
```


Figure 8: Raw Checkmark SVG string

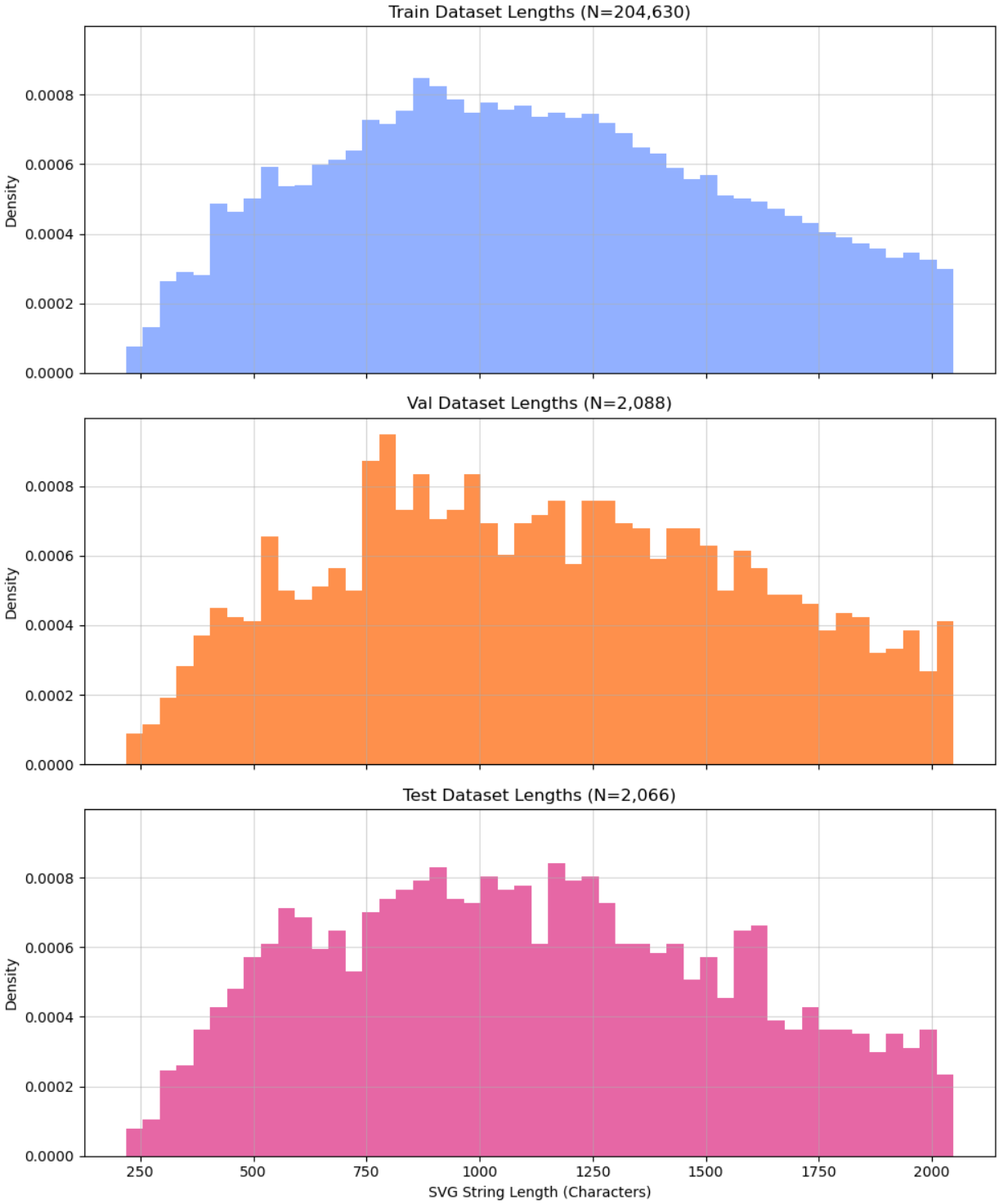


Figure 9: SVG String Length Distributions in Each Subset

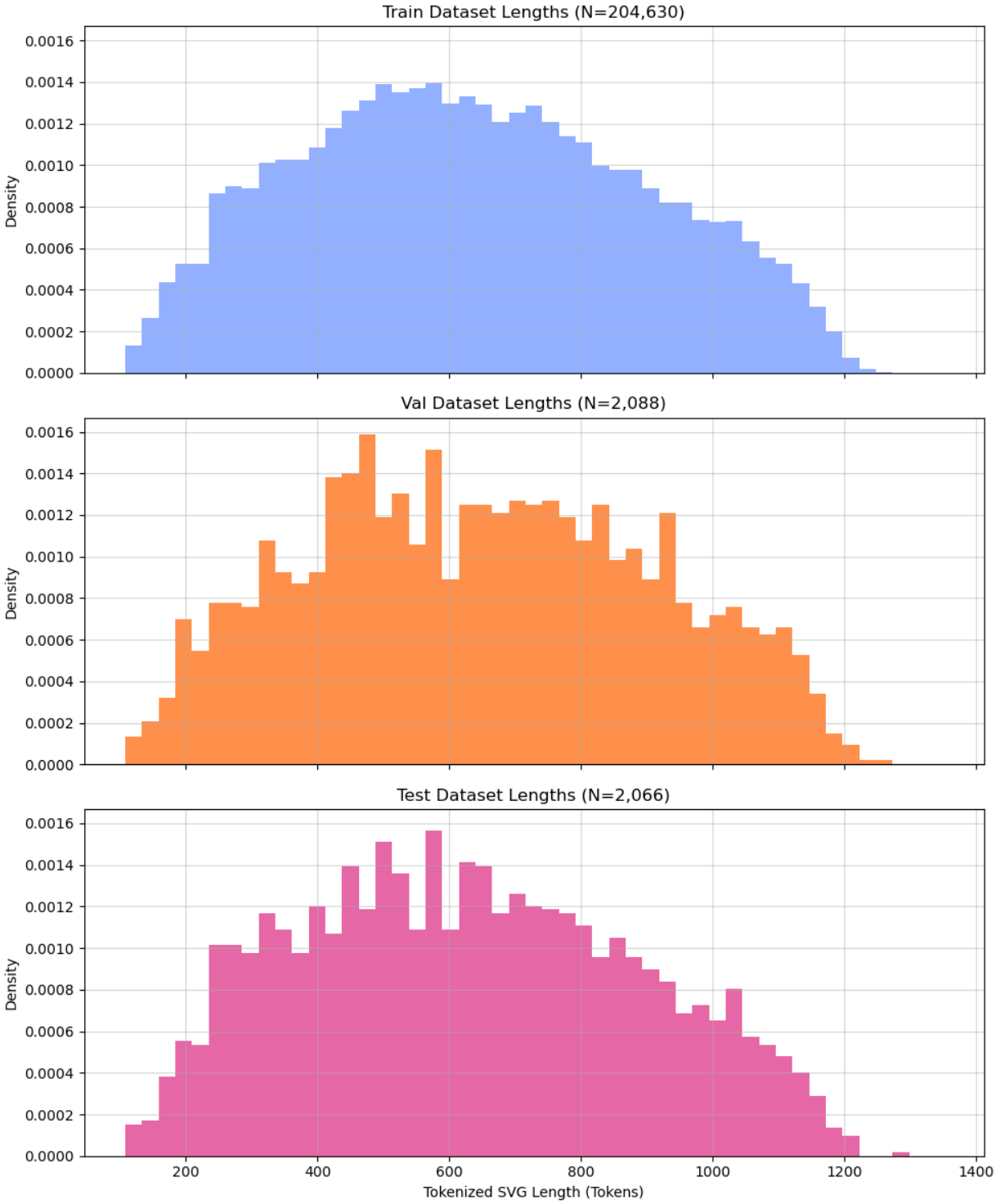


Figure 10: SVG Token Length Distributions in Each Subset

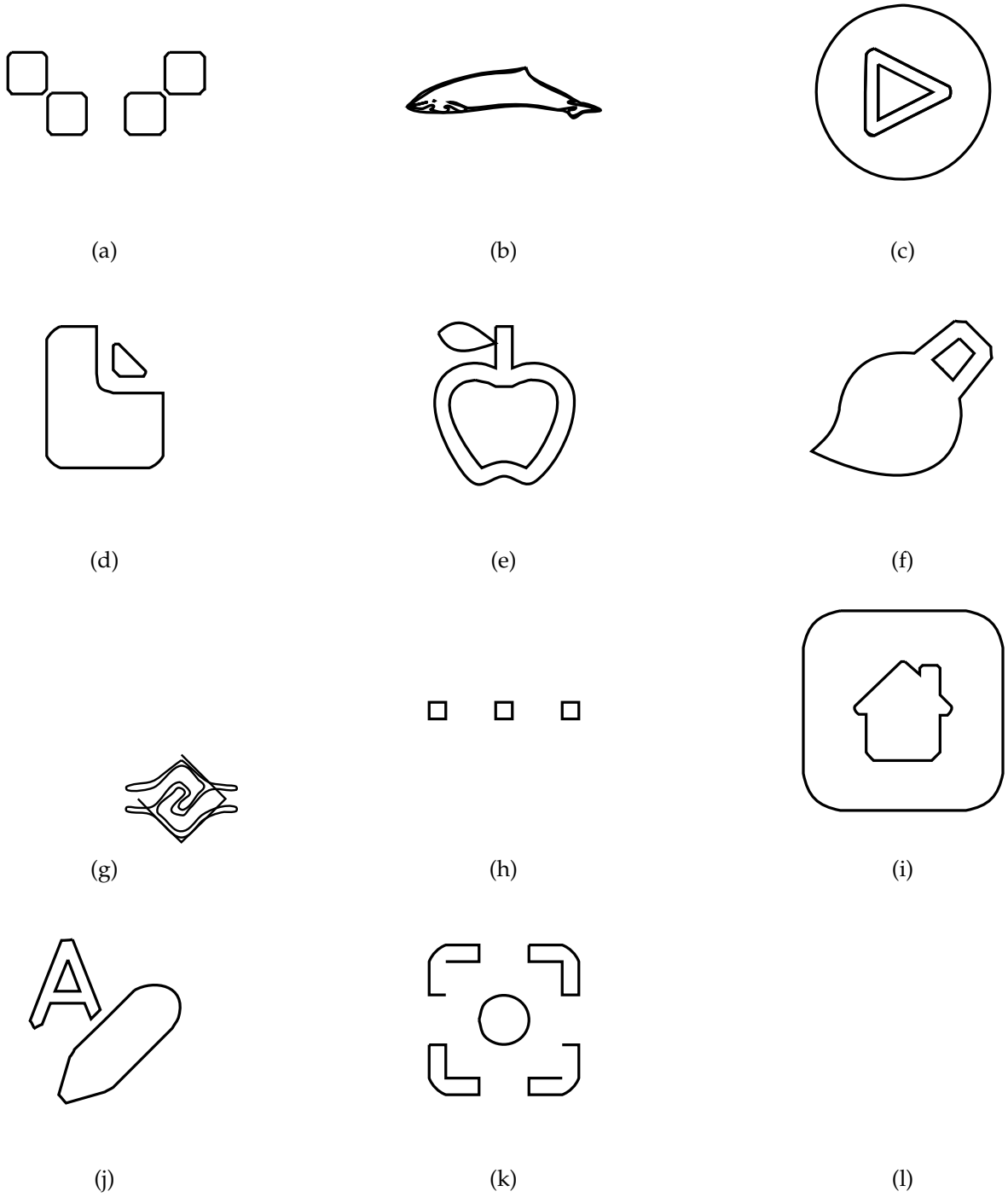


Figure 11: 12 SVGs in training dataset. 11g and 11l are defective (Outside default viewBox, See figures 17 and 18).

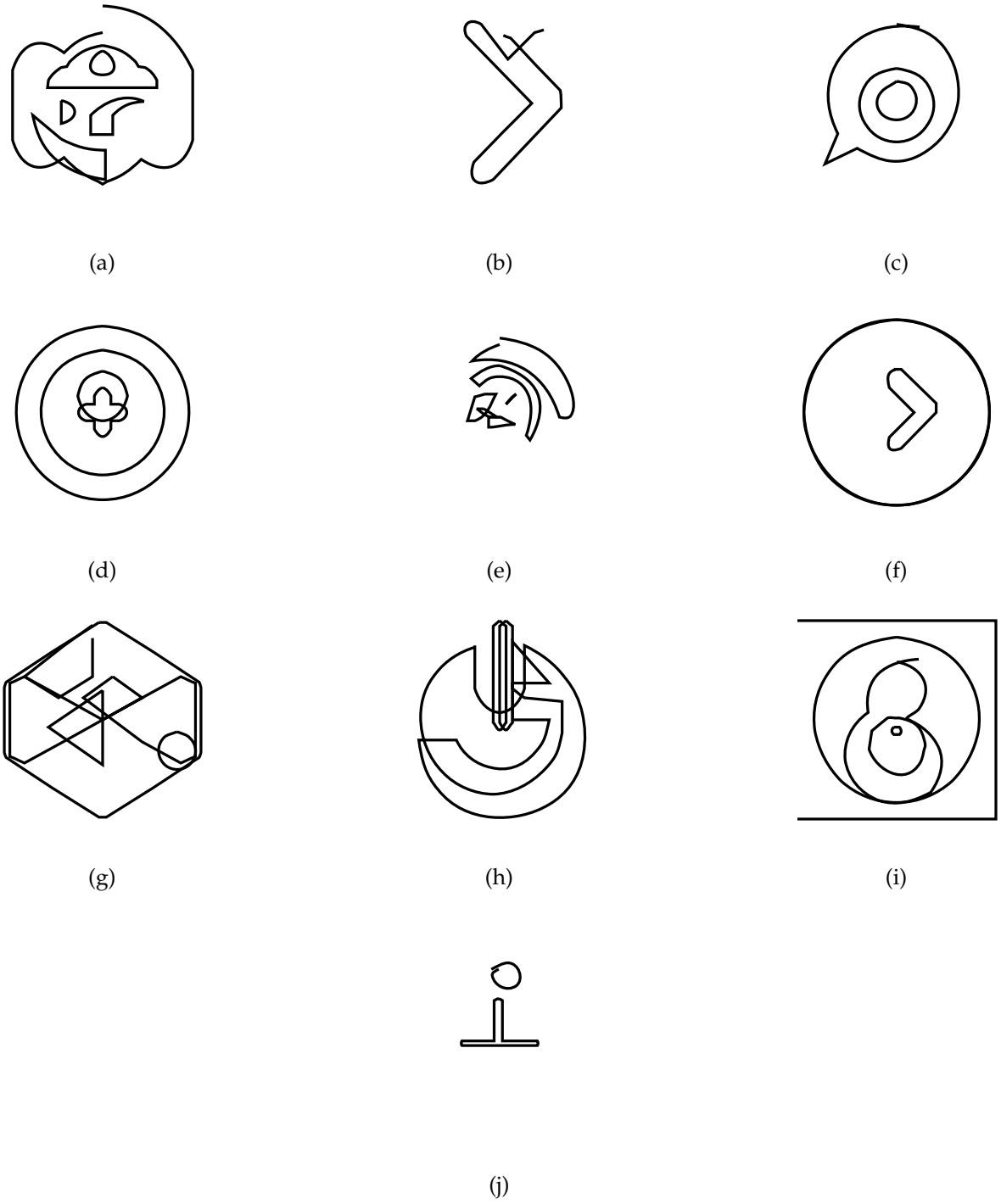


Figure 12: “XL” Model-Generated SVGs, Unconditioned

Generated, post-processed

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"><path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 12.0 1.2 L 12.0 1.2 C 14.3 1.3 16.3 2.0 17.9 3.2 C 19.5 4.4 20.8 6.1 21.8 8.2 L 21.8 12.0 L 21.8 15.8 C 20.9 18.7 18.9 19.7 16.2 17.9 L 16.2 17.9 C 15.0 19.1 14.0 19.8 12.5 20.3 L 12.0 20.6 L 11.5 20.3 C 9.9 19.6 9.0 19.0 7.8 17.7 L 7.8 17.7 C 5.1 19.7 3.1 18.8 2.2 15.8 L 2.2 12.0 L 2.2 8.2 C 3.1 5.3 5.1 4.3 7.8 6.3 L 7.8 6.3 C 9.2 4.9 10.1 4.5 12.0 4.1 L 12.0 4.1"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 12.0 5.5 L 12.0 5.5 C 13.6 5.8 14.4 6.1 15.6 7.3 L 15.9 7.8 L 16.6 7.9 C 17.2 8.2 17.6 8.7 17.9 9.3 L 17.9 10.1 L 12.0 10.1 L 6.0 10.1 L 6.1 9.3 C 6.4 8.7 7.0 8.2 7.5 7.9 L 8.1 7.8 L 8.4 7.3 C 9.6 6.1 10.4 5.8 12.0 5.5 L 12.0 5.5"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 12.0 6.1 L 12.0 6.1 C 13.2 6.5 14.1 8.7 12.0 8.7 C 9.8 8.7 10.7 6.5 12.0 6.1 L 12.0 6.1"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 7.5 11.5 L 7.5 11.5 C 9.4 11.9 9.5 13.5 7.5 14.0 L 7.5 11.5"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.5 11.6 L 16.5 11.6 C 15.0 11.7 14.2 12.0 13.1 13.1 L 13.1 15.2 L 10.7 15.2 L 10.7 13.1 C 12.1 11.7 14.3 10.6 16.5 11.6 L 16.5 11.6"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 7.5 15.7 L 7.5 15.7 C 9.1 16.5 10.3 16.9 12.3 16.9 L 12.3 20.1 C 10.1 19.8 8.3 19.1 6.9 17.9 C 5.6 16.8 4.8 15.2 4.4 13.1 L 7.5 15.7"> </path> </svg>

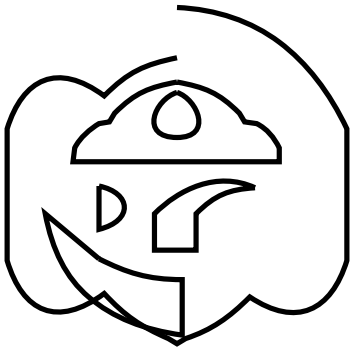
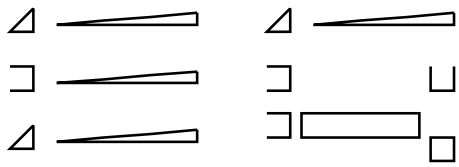
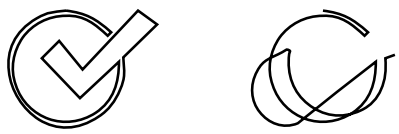


Figure 13: Unconditioned “XL” Model-Generated SVG #1.

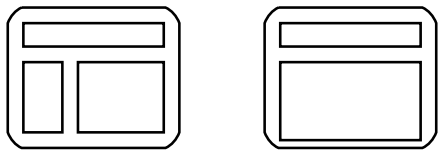
(a)



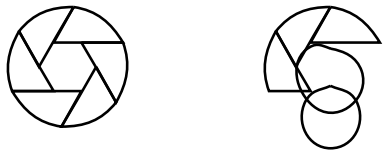
(b)



(c)



(d)



(e)

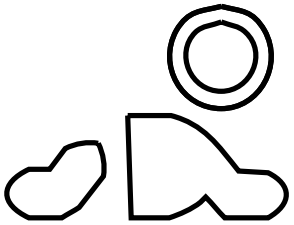
Figure 14: "XL" Model-Generated SVGs.
Conditioned on left, generated right.

Prefix

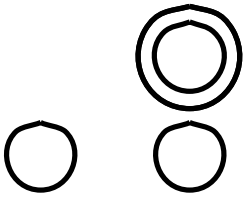

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"><path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 5.7 L 16.3 5.7 C 17.5 6.0 18.1 5.9 18.8 7.1 C 19.9 9.1 18.5 11.6 16.3 11.6 C 14.0 11.6 12.6 9.1 13.7 7.1 C 14.4 5.9 15.1 6.0 16.3 5.7 L 16.3 5.7"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 5.7 L 16.3 5.7 C 17.5 6.0 18.1 5.9 18.8 7.1 C 19.9 9.1 18.5 11.6 16.3 11.6 C 14.0 11.6 12.6 9.1 13.7 7.1 C 14.4 5.9 15.1 6.0 16.3 5.7 L 16.3 5.7"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 6.6 L 16.3 6.6 C 16.8 6.8 17.3 6.8 17.7 7.2 C 19.0 8.4 18.0 10.6 16.3 10.6 C 14.5 10.6 13.6 8.4 14.8 7.2 C 15.2 6.8 15.8 6.8 16.3 6.6 L 16.3 6.6"></path> <path fill="none" stroke
```


Completion, post-processed


```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"><path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 5.7 L 16.3 5.7 C 17.5 6.0 18.1 5.9 18.8 7.1 C 19.9 9.1 18.5 11.6 16.3 11.6 C 14.0 11.6 12.6 9.1 13.7 7.1 C 14.4 5.9 15.1 6.0 16.3 5.7 L 16.3 5.7"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 5.7 L 16.3 5.7 C 17.5 6.0 18.1 5.9 18.8 7.1 C 19.9 9.1 18.5 11.6 16.3 11.6 C 14.0 11.6 12.6 9.1 13.7 7.1 C 14.4 5.9 15.1 6.0 16.3 5.7 L 16.3 5.7"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 6.6 L 16.3 6.6 C 16.8 6.8 17.3 6.8 17.7 7.2 C 19.0 8.4 18.0 10.6 16.3 10.6 C 14.5 10.6 13.6 8.4 14.8 7.2 C 15.2 6.8 15.8 6.8 16.3 6.6 L 16.3 6.6"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 7.7 12.4 L 7.7 12.4 C 8.1 12.6 8.7 12.6 9.1 12.9 C 10.3 14.0 9.5 16.3 7.7 16.3 C 5.9 16.3 5.1 14.0 6.3 12.9 C 6.7 12.6 7.2 12.6 7.7 12.4 L 7.7 12.4"> </path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 16.3 12.4 L 16.3 12.4 C 16.8 12.6 17.3 12.6 17.7 12.9 C 18.9 14.0 18.1 16.3 16.3 16.3 C 14.5 16.3 13.7 14.0 14.9 12.9 C 15.3 12.6 15.9 12.6 16.3 12.4 L 16.3 12.4"> </path> </svg>
```

(a) Conditioned



(b) Completed

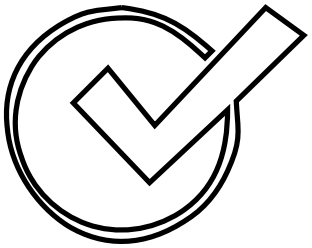
Figure 15: Conditioned “XL” Model-Generated SVG #1.

Prefix

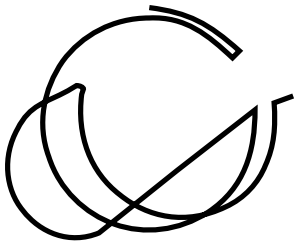
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 10.1 5.3 L 10.1 5.3 C 12.4 5.6 13.5 6.2 15.3 7.8 L 14.9 8.2 C 13.5 6.9 12.2 5.8 10.1 5.9 C 8.0 5.9 6.2 6.9 5.1 8.4 C 4.0 10.0 3.6 12.0 4.3 13.9 C 4.9 15.8 6.4 17.2 8.1 17.8 C 9.8 18.4 11.8 18.2 13.5 17.1 C 15.6 15.7 16.2 13.6 16.2 11.2 L 11.7

Completion, post-processed

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px" <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 10.1 5.3 L 10.1 5.3 C 12.4 5.6 13.5 6.2 15.3 7.8 L 14.9 8.2 C 13.5 6.9 12.2 5.8 10.1 5.9 C 8.0 5.9 6.2 6.9 5.1 8.4 C 4.0 10.0 3.6 12.0 4.3 13.9 C 4.9 15.8 6.4 17.2 8.1 17.8 C 9.8 18.4 11.8 18.2 13.5 17.1 C 15.6 15.7 16.2 13.6 16.2 11.2 L 11.7 14.7 L 7.2 18.3 C 5.5 19.0 3.8 18.3 2.8 17.0 C 1.8 15.8 1.6 13.9 2.5 12.3 C 3.2 10.9 4.1 10.8 5.4 10.1 L 5.9 9.8 C 6.1 9.8 6.3 9.9 6.3 10.0 L 6.2 10.3 C 5.9 12.7 6.7 14.8 8.4 16.1 C 10.0 17.4 12.2 17.9 14.5 16.8 C 15.6 16.3 16.4 15.4 16.8 14.4 C 17.3 13.3 17.4 12.2 17.3 10.8 L 18.4 10.4"> </path> </svg>



(a) Conditioned



(b) Completed

Figure 16: Conditioned “XL” Model-Generated SVG #3.

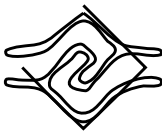


Figure 11g
Small, uncentered SVG when displayed in this document.

Figure 11g
Blank SVG/PNG when viewed anywhere else.

Cleaned SVG:

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 28.0 22.0 L 31.2 25.2 L 34.4 28.4 L 31.2 31.6 L 28.0 34.7 L 24.8 31.6 L 21.7 28.4"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 28.0 22.8 L 28.0 22.8 C 29.4 23.7 30.7 25.5 32.2 26.1 C 32.8 26.3 33.5 26.3 34.2 26.4 C 34.7 26.5 35.4 26.4 35.9 26.6 C 36.1 26.7 36.1 27.1 35.9 27.1 C 34.7 27.4 33.4 26.7 32.2 27.1 C 30.9 27.5 30.2 28.9 29.2 29.8 C 28.9 30.1 28.6 30.8 28.2 30.8 C 24.3 30.4 28.5 28.3 28.5 27.5 C 28.5 27.1 28.0 26.8 27.6 26.8 C 27.2 26.8 27.0 27.4 26.7 27.6 C 25.9 28.4 24.4 29.0 24.5 30.2 C 24.5 30.6 25.0 30.8 25.3 31.1 C 26.2 31.9 26.9 33.2 28.0 33.1 C 29.2 33.1 29.9 31.7 30.8 30.9 C 31.9 30.0 34.5 29.0 35.9 29.6 C 36.1 29.7 36.1 30.1 35.9 30.2 C 35.4 30.4 34.7 30.3 34.2 30.3 C 33.0 30.5 31.9 30.9 31.0 31.6 C 30.1 32.4 29.3 34.0 28.0 34.0 C 26.4 34.0 25.0 31.2 23.6 30.7 C 22.5 30.2 21.2 30.6 20.1 30.2 C 19.9 30.1 19.9 29.7 20.1 29.6 C 20.6 29.5 21.2 29.7 21.7 29.7 C 22.3 29.7 23.0 29.8 23.6 29.6 C 25.1 29.1 26.0 26.2 27.6 26.0 C 28.0 25.9 28.3 26.4 28.6 26.6 C 30.4 28.2 27.9 28.0 27.6 29.5 C 27.5 29.7 27.9 30.0 28.2 30.0 C 29.3 29.7 29.8 28.4 30.5 27.6 C 30.8 27.3 31.3 27.0 31.3 26.6 C 31.3 26.2 30.8 25.9 30.5 25.7 C 29.8 24.9 29.1 23.6 28.0 23.6 C 26.6 23.7 25.3 26.1 24.1 26.7 C 23.4 27.1 20.9 27.5 20.1 27.1 C 20.0 27.0 19.9 26.7 20.1 26.6 C 22.0 25.8 23.3 26.6 25.0 25.1 C 26.0 24.3 27.0 23.5 28.0 22.8 L 28.0 22.8"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 19.9 26.3 L 25.3 26.3 L 30.7 26.3 L 36.1 26.3 L 36.1 30.5 L 30.7 30.5 L 25.3 30.5 L 19.9 30.5"></path></svg>

Figure 17: Rendering error of 11g.

Figure 11l
Blank SVG when displayed in this document.

Figure 11l
Blank SVG/PNG when viewed anywhere else.

Cleaned SVG:

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0.0 0.0 24.0 24.0" height="200px" width="200px"> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 925.9 695.1 L 930.7 695.1 L 935.4 695.1 L 935.4 699.8 L 935.4 704.5 L 930.7 704.5 L 925.9 704.5 L 925.9 699.8 L 925.9 695.1"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 938.1 695.1 L 942.8 695.1 L 947.5 695.1 L 947.5 699.8 L 947.5 704.5 L 942.8 704.5 L 938.1 704.5 L 938.1 699.8 L 938.1 695.1"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 925.9 707.2 L 930.7 707.2 L 935.4 707.2 L 935.4 712.0 L 935.4 716.7 L 930.7 716.7 L 925.9 716.7 L 925.9 712.0 L 925.9 707.2"></path> <path fill="none" stroke="black" stroke-width=".3" stroke-opacity="1.0" filling="0" d="M 938.1 707.2 L 942.8 707.2 L 947.5 707.2 L 947.5 712.0 L 947.5 716.7 L 942.8 716.7 L 938.1 716.7 L 938.1 712.0 L 938.1 707.2"></path></svg>

Figure 18: Rendering error of 11l.

'<|endoftext|>': 0, '<|pad|>': 1, '"': 2, '-': 3, '.': 4, '/': 5, '0': 6, '1': 7, '2': 8, '3': 9, '4': 10, '5': 11, '6': 12, '7': 13, '8': 14, '9': 15, ':': 16, '<': 17, '=': 18, '>': 19, 'B': 20, 'C': 21, 'L': 22, 'M': 23, 'a': 24, 'b': 25, 'c': 26, 'd': 27, 'e': 28, 'f': 29, 'g': 30, 'h': 31, 'i': 32, 'k': 33, 'l': 34, 'm': 35, 'n': 36, 'o': 37, 'p': 38, 'r': 39, 's': 40, 't': 41, 'v': 42, 'w': 43, 'x': 44, 'y': 45, 'th': 46, 'ke': 47, 'oke': 48, 'pa': 49, 'roke': 50, 'st': 51, 'stroke': 52, '12': 53, '20': 54, '10': 55, 'fi': 56, 'll': 57, 'path': 58, 'fill': 59, '14': 60, '15': 61, '11': 62, '13': 63, '16': 64, '17': 65, '18': 66, '19': 67, 'dth': 68, 'idth': 69, 'width': 70, '21': 71, '22': 72, 'ac': 73, 'bl': 74, 'ci': 75, 'in': 76, 'ne': 77, 'no': 78, 'opa': 79, 'ty': 80, 'fillin': 81, 'ack': 82, 'black': 83, 'city': 84, 'none': 85, 'opacity': 86, 'filling': 87, '200': 88, 'sv': 89, 'svg': 90, '24': 91, 'ht': 92, 'px': 93, 'ww': 94, '200px': 95, '2000': 96, 'Bo': 97, 'ei': 98, 'ew': 99, 'ght': 100, 'hei': 101, 'iew': 102, 'ln': 103, 'mln': 104, 'or': 105, 'tp': 106, 'view': 107, 'w3': 108, 'xmln': 109, 'http': 110, 'www': 111, 'Box': 112, 'height': 113, 'org': 114, 'viewBox': 115, 'xmlns': 116, '23': 117, '25': 118, '26': 119, '27': 120, '28': 121, '29': 122, '30': 123, '33': 124, '34': 125, '35': 126, '36': 127, '39': 128, '40': 129, '38': 130, '31': 131, '37': 132,
⋮
'888': 992, '1024': 993, '1185': 994, '796': 995, '904': 996, '896': 997, '1009': 998, '756': 999

Figure 19: Trained Tokenizer Vocabulary

```python
class Head(nn.Module):
        """one head of self-attention"""

        def __init__(self, head_size, d_model, block_size):
                super().__init__()
                self.key = nn.Linear(d_model, head_size, bias=False)
                self.query = nn.Linear(d_model, head_size, bias=False)
                self.value = nn.Linear(d_model, head_size, bias=False)
                self.head_size = head_size

                # 'tril' is the lower triangular matrix for masking
                self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))

        def forward(self, x):
                B, T, C = x.shape  # Batch, sequence length, embedding dimensionality (d_model)

                k = self.key(x)
                q = self.query(x)
                v = self.value(x)

                # Compute attention scores
                wei = q @ k.transpose(-2, -1) * (self.head_size**-0.5)

                # Mask out future tokens by replacing 0s in the tril matrix with -infinity
                wei = wei.masked_fill(self.tril[:T, :T] == 0, float("-inf"))

                output = F.softmax(wei, dim=-1) @ v

                return output
```

Figure 20: Head Class

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, n_heads, d_model, block_size):
        super().__init__()

        head_size = d_model // n_heads

        self.heads = nn.ModuleList(
            [Head(head_size, d_model, block_size) for _ in range(n_heads)]
        )

        self.proj = nn.Linear(head_size * n_heads, d_model)

    def forward(self, x):

        # Runs input through each attention head, and glues them together side-by-side
        x = torch.concat([head(x) for head in self.heads], dim=-1)

        # Mixes the (head_size x n_heads) head outputs back to one d_model dimensional layer
        x = self.proj(x)

        return x
```

Figure 21: MultiHeadAttention Class

```python
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff=None):
        super().__init__()

        if d_ff == None:
            d_ff = 4 * d_model

        self.layer = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(d_ff, d_model)
        )

    def forward(self, x):
        return self.layer(x)
```

Figure 22: FeedForward Class

```python
class TransformerBlock(nn.Module):
    def __init__(self, n_heads, d_model, d_ff, block_size):
        super().__init__()

        self.MHA = MultiHeadAttention(
            n_heads=n_heads, d_model=d_model, block_size=block_size
        )
        self.ff = FeedForward(d_model=d_model, d_ff=d_ff)
        self.ln_1 = nn.LayerNorm(d_model)
        self.ln_2 = nn.LayerNorm(d_model)

    def forward(self, x):

        x = x + self.MHA(self.ln_1(x))
        x = x + self.ff(self.ln_2(x))

        return x
```

Figure 23: TransformerBlock Class

```python
class CustomTransformer(nn.Module):
    def __init__(self, vocab_size, block_size, n_layers, n_heads, d_model, d_ff):
        super().__init__()
        self.vocab_size = vocab_size
        self.block_size = block_size
        self.n_layers = n_layers
        self.n_heads = n_heads
        self.d_model = d_model
        self.d_ff = d_ff
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.positional_embedding = nn.Embedding(block_size, d_model)
        self.blocks = nn.Sequential(
            *[TransformerBlock(n_heads, d_model, d_ff, block_size) for _ in range(n_layers)]
        )
        self.ln_f = nn.LayerNorm(d_model)
        self.proj = nn.Linear(d_model, vocab_size)

    def generate(self, idx, max_new_tokens, temperature=0.8):
        for _ in range(max_new_tokens):
            # Crop to block_size if too long
            if idx.size(1) <= self.block_size:
                idx_continue = idx
            else:
                idx_continue = idx[:, -self.block_size :]
            logits, _ = self(idx_continue)
            logits = logits[:, -1, :] / temperature
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.concat((idx, idx_next), dim=1)
        return idx

    def forward(self, idx, targets=None):
        sequence_length = idx.shape[1]
        te = self.token_embedding(idx)
        pe = self.positional_embedding(torch.arange(sequence_length, device=idx.device))
        x = te + pe  # input_embedding
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.proj(x)
        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B * T, C)
            targets = targets.view(B * T)
            loss = F.cross_entropy(logits, targets)
        return logits, loss
```

Figure 24: CustomTransformer Class

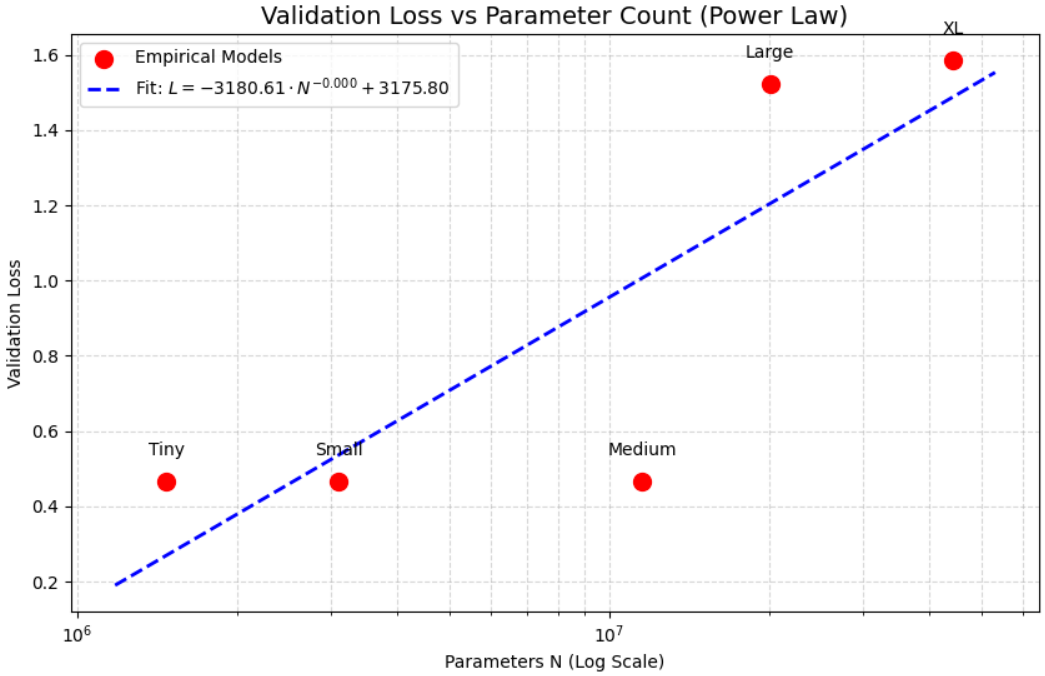


Figure 25: Standard Loss Graphs, Fixed Learning Rate = 0.01

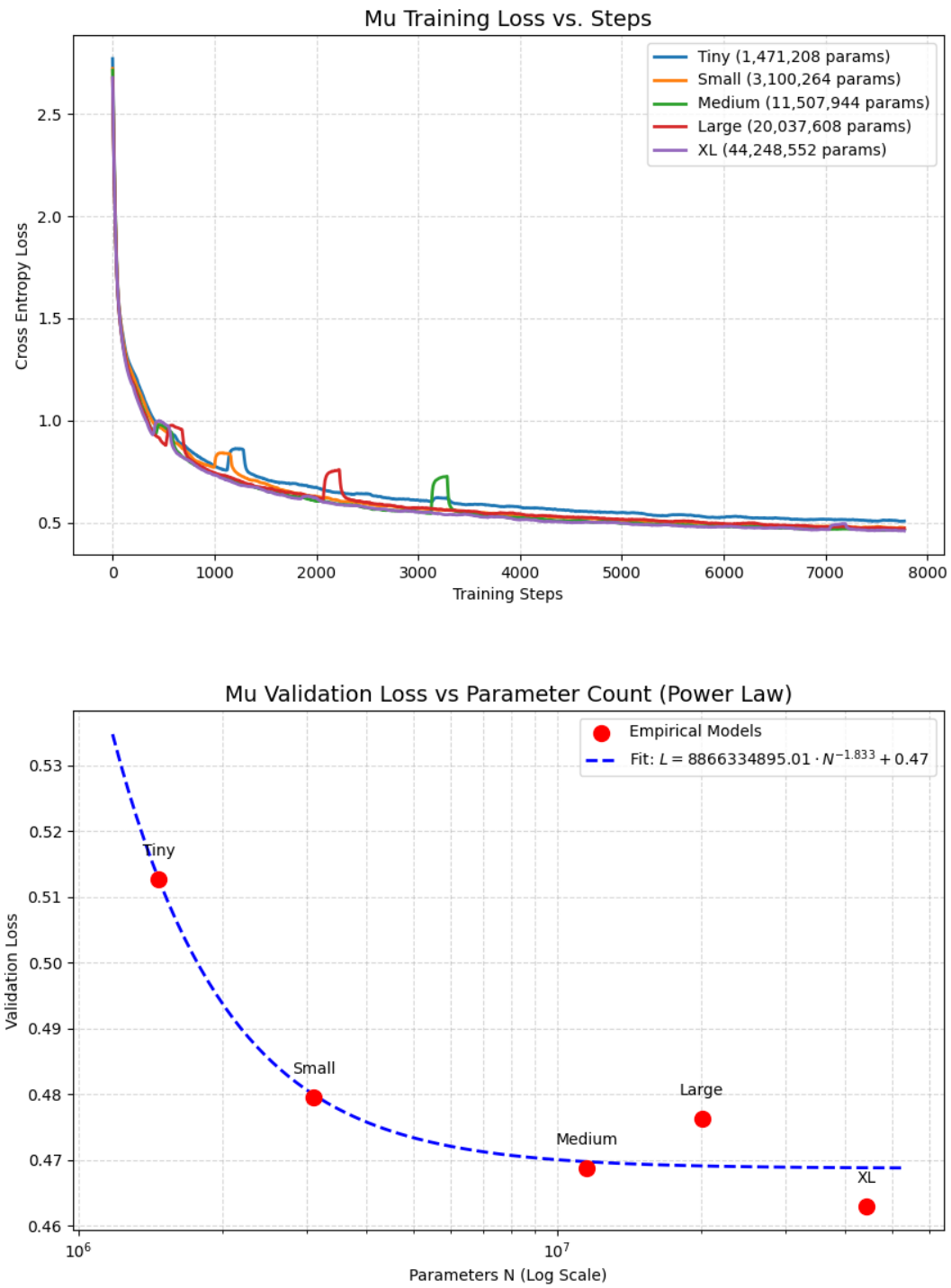


Figure 26: μ**P** Loss Graphs, Scaling Learning Rate = 0.01

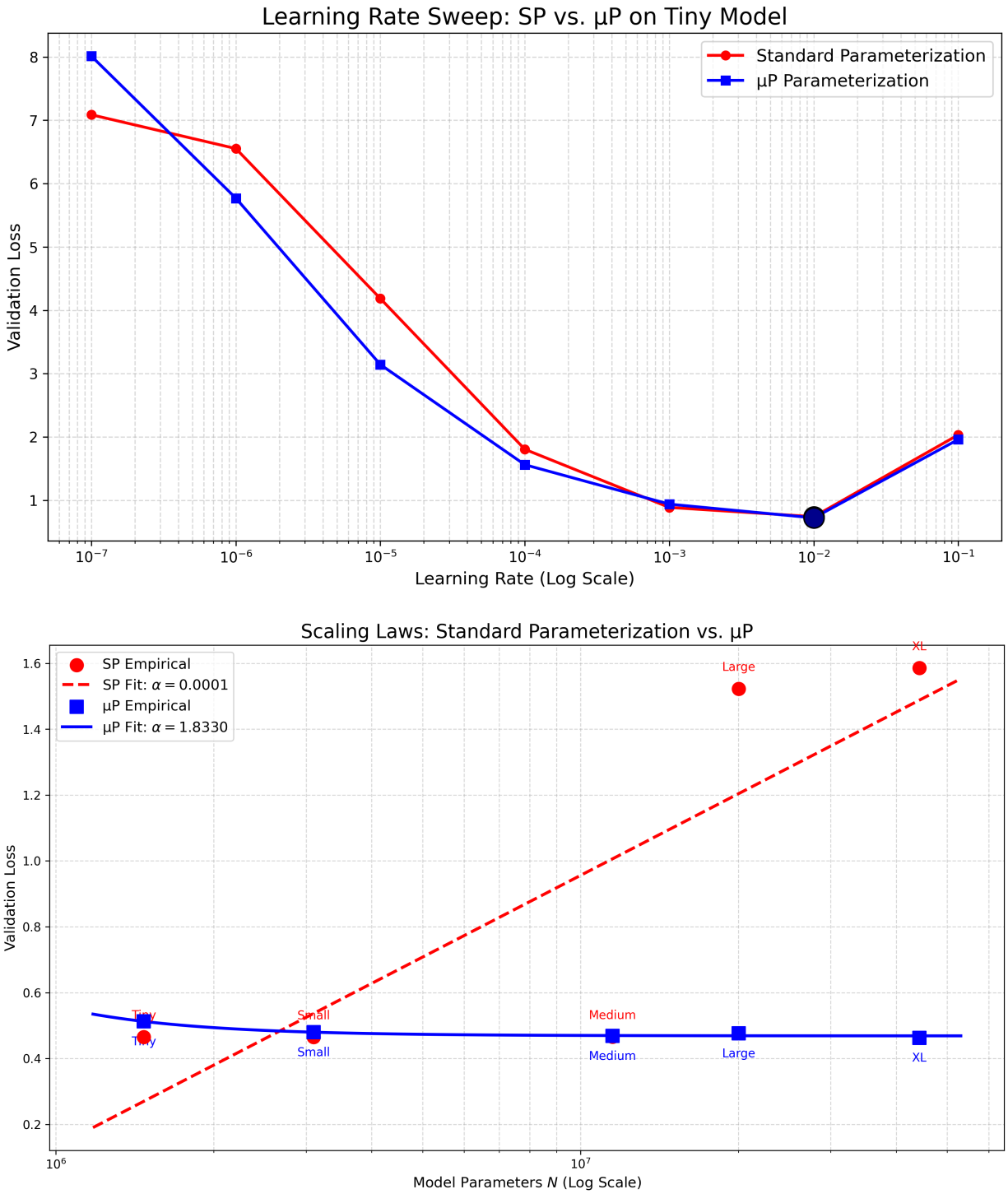


Figure 27: Comparison Between Standard and μ**P** Parameterized Models